ENSEIRB-MATMECA 1ère année Informatique

Royal Dédale
Rapport du projet

Groupe I1 P5 - Équipe n°8

Ilan Vargas & Achraf Safi

Encadré par Tristan Riehs

19 décembre 2025



Table des matières

Introduction & Architecture **2**

1 Fondation **4**
1.1 Code source . . . 4
1.1.1 directions . . . 4
1.1.2 positions . . . 4
1.1.3 ressources . . . 4
1.1.4 poches . . . 5
1.2 Tests et validation . . . 5
1.2.1 Tests sur les directions et les ressources . . . 5
1.2.2 Tests sur les positions . . . 6
1.2.3 Tests sur les poches . . . 6
1.2.4 Conclusion sur la validité . . . 6
1.3 Conclusion . . . 6

2 Implémentation des tuiles **7**
2.1 Abstraction et interface du module `tile` . . . 7
2.2 Structure interne d'une tuile . . . 7
2.2.1 Définition statique des tuiles . . . 7
2.3 Représentation du plateau de jeu . . . 8
2.4 Opérations sur les tuiles . . . 8

3 Implémentation du jeu **10**
3.1 Manoir et joueur . . . 10
3.2 Boucle du jeu . . . 10
3.3 Exécution . . . 13
3.3.1 Le Makefile . . . 13
3.3.2 Paramètres d'exécution . . . 13
3.4 Visualisation graphique : TUI . . . 14

4 Limites et améliorations **15**
4.1 Achievement 1 : Rotation des tuiles . . . 15
4.2 Achievement 2 : Tuiles objectives . . . 16
4.2.1 Changement en game_state . . . 16
4.2.2 Modification de la boucle pour les `target_tiles` . . . 16
4.3 Ajouter une intelligence au joueur . . . 16
4.3.1 path_target_finder . . . 16
4.3.2 Fonction récursive de recherche de chemin . . . 17
4.3.3 Complexité . . . 17
4.3.4 Accommodation des changements dans la boucle de jeu . . . 18

4.4 Achievement 3 : Équipements fonctionnnels 18
4.5 Nouvelle boucle de jeu & architecture 19

Remarques et conclusion **20**

Annexes **21**

Introduction

Le projet de programmation Royal Dédale consiste à implémenter un jeu d'exploration se déroulant dans un manoir représenté sous la forme d'une grille rectangulaire de taille fixe. À l'exception de la tuile de départ et de la tuile d'arrivée, le manoir est initialement composé uniquement d'espaces vides, correspondant à des positions encore inconnues du joueur.

Le joueur débute la partie sur la tuile de départ et doit atteindre la tuile d'arrivée en explorant progressivement le manoir. Cette exploration repose sur l'ouverture de tuiles adjacentes aux positions déjà découvertes, puis sur le déplacement du joueur de case en case. Les tuiles sont ainsi placées dynamiquement au cours de la partie, ce qui modifie progressivement la configuration du manoir et confère au jeu sa structure labyrinthique.

Chaque tuile possède des caractéristiques propres, notamment des coûts de traversée et d'ouverture selon chacune des directions cardinales. Ces contraintes, associées à une gestion limitée des ressources et à un nombre maximal de tours, conditionnent les choix du joueur et influencent directement sa progression.

L'objectif de ce projet est de concevoir et d'implémenter une solution logicielle respectant strictement les interfaces abstraites imposées par le sujet, tout en garantissant le bon déroulement d'une partie dans le cadre de la version de base du jeu.

Afin de répondre aux contraintes du sujet, une architecture logicielle modulaire a été mise en place. La section suivante présente l'organisation générale du programme.

Architecture du programme

Nous détaillerons l'architecture du programme C de notre version initiale de *Royal Dédale*

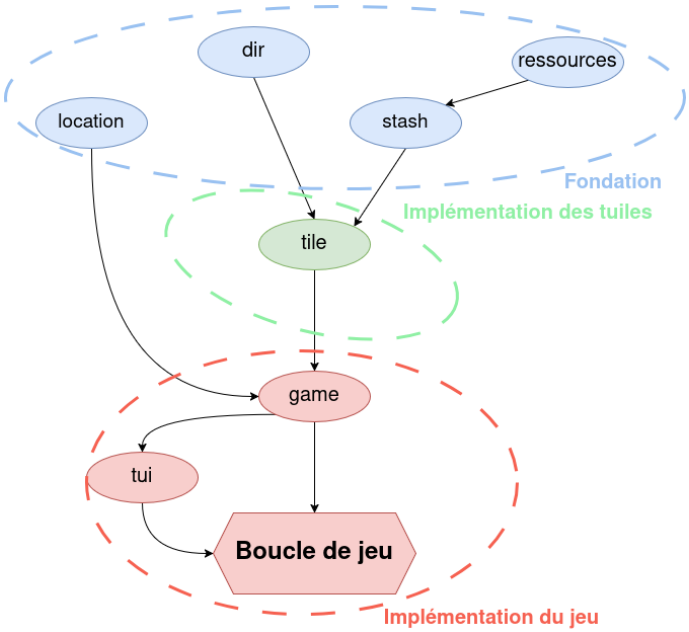


FIGURE 1 – Architecture de la version initiale du projet

La figure 1 présente les différents modules de notre programme ainsi que leurs interdépendances. Chaque flèche indique qu'un module dépend de celui dont elle part, ainsi que de l'ensemble des modules dont ce dernier dépend. Nous pouvons regrouper ces modules en trois ensembles distincts.

- La **fondation**. Où on implémente les "types" (structures et énumérations) fondamentaux pour notre programme : *localisation*, *direction*(*dir*), *resource* et *stash*(*poche*) ; et leurs fonctions associées. Ces conceptes servent de base à laquelle s'appuient les modules suivants.
- L'**implémentation des tuiles**. Avec les modules précédents nous créons les tuiles et leurs opérations. C'est la composition du manoir implémenté par la suite qui une mécanique fondamentale de la partie.
- L'**implémentation du jeu**. C'est là que nous créons le manoir, les fonctions pour le manipuler, des fonctions pour l'afficher en forme d'interface textuelle pour combiner tout ça dans une boucle qui nous permettra de jouer le jeu. Dans cette étape nous combinons tout notre code précédent pour créer finalement le jeu.

Cette architecture donne un jeu fonctionnel mais basique en termes d'optimalité ou de jouabilité, nous verrons les limites et les améliorations proposées sur ce programme. Mais avant ça, nous commençons par détailler la fondation de cette architecture.

1 Fondation

Nous détaillons maintenant les "types" et leurs opérations, utilisés comme base de ce projet.

1.1 Code source

1.1.1 directions

On définit un type énuméré *dir_t* afin de gérer les différentes directions cardinales (nord, sud, west, est) d'une manière simple et claire tout au long du projet.

Le module implémente les fonctions `dir_to_string` et `dir_to_short_string` qui renvoient la représentation d'une direction en chaîne de caractères. De plus, il implémente la fonction `dir_opposite` qui permet d'obtenir la direction opposée à celle fournie en entrée.

1.1.2 positions

On définit la position par une structure *location_t* contenant un couple de coordonnées (x, y) correspondant respectivement à l'abscisse et l'ordonnée d'une position dans le manoir. Une fonction `location` est fournie afin de générer cette structure à partir du couple de coordonnées fournies.

Le module inclut les fonctions nécessaires pour manipuler les positions dans le manoir : `location_is_valid` verifie si une position est bien dans le manoir, `location_equal` verifie si deux position sont égales et `location_is_adjacent` vérifie qu'une position est adjacente à une autre.

Les deux dernières fonctions renvoient faux si une des positions n'est pas valide.

1.1.3 ressources

On définit le type énuméré *resource_t* qui représente l'ensemble des types de ressources dans le jeu dont le nombre est fixé à MAX_RESOURCES = 10, cette implémentation simplifie par suite le stockage et la manipulation des ressources du joueur dans une poche, décrite dans la section suivante.

Parmi les ressources différentes disponibles, la plus notable est les *steps*(*pas*). Jusqu'à la version améliorée, pour chaque mouvement ou placement de tuile on consomme une *step* et donc on a besoin d'avoir au minimum une *step* pour faire une action pendant son tour sinon le joueur est coincé.

L'implémentation inclut des fonctions qui permettent de représenter la ressource en chaîne de caractère, soit `resource_to_string` et `resource_to_short_string`.

1.1.4 poches

Après avoir défini les types de ressources, nous implémentons maintenant le module *stash* (poche) qui constitue le système de gestion des ressources du joueur.

La structure *stash_t*, représente une poche de ressources. Il s'agit d'un tableau dont la taille correspond exactement au nombre de ressources en jeu où chaque case du tableau correspond à la quantité de la ressource associée à son indice, selon *resource_t*.

Ce fichier regroupe l'ensemble des opérations sur les poches :

— `stash_add`, l'addition de deux poches.

— `stash_sub`, la soustraction d'une poche à une autre (si la soustraction donne un entier négatif la ressource résultante sera nulle)

— `stash_contains`, verifie si une poche est contenue dans une poche mère (on vérifie donc à ce que toutes les ressources soient inférieures ou égales à celles de la poche mère)

— `stash_to_string`, représentation en chaîne de caractère

1.2 Tests et validation

Dans cette partie on s'intéressera à l'implémentation des tests et comment ils permettent de valider la correction des fonctions.

Généralement, les fonctions du code source sont testées sur plusieurs entrées ce qui demande la création de trois tableaux, un tableau pour les entrées, un pour les sorties et un autre contenant les solutions attendues pour comparer la sortie à la solution. Un exemple de ces tests sera fourni dans ce qui suit.

1.2.1 Tests sur les directions et les ressources

```
1 TESTS ON resource.c
2 Tests on resource_to_string
3 PASSED  input:   0     output: NOTHING
4 PASSED  input:   1     output: STEP
5 PASSED  input:   5     output: SILK
6 PASSED  input:   9     output: OIL
7 PASSED  input:  10     output: CRYSTAL
```

Code 1 – Exemple des tests sur resource_to_string

Les tests sur ce module servent à vérifier la sortie des fonctions de représentation des "types" en chaîne de caractère, on compare les chaînes de sortie et celle attendue.

D'autre part, le test `dir_oposite` demeure simple grâce au type énuméré des directions, il suffit de vérifier l'égalité entre la sortie et la solution autant que deux entiers.

1.2.2 Tests sur les positions

Afin de valider les tests sur les fonctions manipulant les positions du manoir, appelons les fonction sur des positions différentes dans le but de tester tous les cas pertinents, notamment les positions dedans et dehors le manoir pour tester la validité de la position dans ce dernier ; des couples égaux adjacents ou pas pour les fonctions qui comparent deux positions.

1.2.3 Tests sur les poches

Avant de procéder aux tests, une fonction est implémentée afin de générer des poches dont les ressources sont des valeurs aléatoires inférieures à 10. Cette fonction permettra ensuite d'observer facilement les quantités de chaque ressource de la poche retournée en les comparant à celles des poches passées en argument, et ainsi de déduire si les opérations effectuées sont bien valides.

1.2.4 Conclusion sur la validité

Même si nous ne pouvons pas tester toutes les combinaisons possibles pour chaque fonction, on déduit qu'on a testé assez de cas pertinents pour assumer que les fonctions sont valides et poser notre programme dessus.

1.3 Conclusion

L'ensemble des fichiers du code source présentés dans cette section ne présente pas de difficultés particulières en termes d'implémentation ou de complexité mais ils sont toutefois essentiels, car ils permettent de définir et de structurer les données utilisées par la suite. Le découpage du code en plusieurs fichiers, associé à la compilation séparée, facilite le travail en équipe. Chaque module peut être développé, modifié ou amélioré indépendamment des autres, tout en respectant les interfaces définies par les fichiers d'en-tête.

Cette organisation permet ainsi d'intégrer différentes implémentations d'un même module sans impacter le reste du programme.

Nous verrons maintenant comment passer à l'étape suivante : l'implémentation des tuiles.

2 Implémentation des tuiles

2.1 Abstraction et interface du module `tile`

La gestion des tuiles est assurée par le module `tile`, dont l'interface est définie dans le fichier `tile.h`. Ce module repose sur une structure abstraite `tile_t`, dont la définition interne est volontairement masquée au reste du programme. Cette abstraction permet de découpler l'utilisation des tuiles de leur implémentation concrète, et garantit que les autres modules du jeu ne dépendent que de l'interface publique.

Grâce à ce choix, il est possible de modifier l'implémentation des tuiles dans `tile.c` (ajout de nouvelles tuiles, modification des coûts d'ouverture et de traverser) sans nécessiter de changements dans le reste du code, tant que les fonctions exposées par `tile.h` conservent le même comportement.

2.2 Structure interne d'une tuile

Dans le fichier `tile.c`, la structure `tile_t` est définie par plusieurs attributs caractérisant une tuile du jeu :

— un `nom`, pour identifier la tuile
— un `coût de traversée` poche, payé à chaque passage du joueur sur la tuile
— un tableau de `coûts d'ouverture` de poches, indexé par direction, indiquant si la tuile peut être ouverte pour chaque direction et le coût
— un `poids`, influence la probabilité d'être généré.

Cette représentation permet de décrire de manière uniforme l'ensemble des tuiles du jeu.

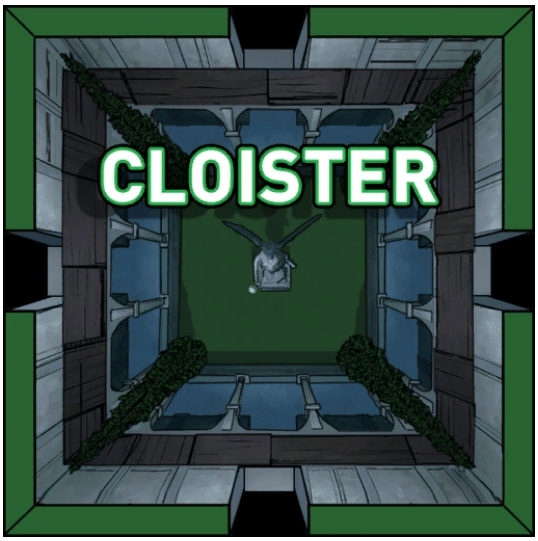
2.2.1 Définition statique des tuiles

Les différentes tuiles du jeu sont définies comme des instances statiques de `tile_t` dans le fichier `tile.c`. Cette approche évite toute allocation dynamique inutile et garantit que les caractéristiques des tuiles sont centralisées en un seul endroit.

On définit, dans notre version, les tuiles spéciales : tuile vide `EMPTY_TILE`, tuile de départ `START_TILE` et tuile d'arrivée `EXIT_TILE`. Puis les tuiles à placé pendant la partie :

— `CLOISTER_TILE`, s'ouvre dans toutes les directions, a un poid égale à 2 (plus de chance d'être généré)
— `EW_CORRIDOR_TILE & NS_CORRIDOR_TILE`, tuiles couloirs, une par orientations
— Quatre tuiles terraces, on implémente une une tuile par direction

Pour l'instant, tout coût d'ouverture ou de traversage est égal à 1 *step*(*pas*). Si une tuile n'est pas ouverte selon une direction, le coût d'ouverture de telle direction pour la tuile est une poche contenant `UINT_MAX` pour la ressource step, c'est-à-dire l'entier maximal en C.




```c
struct tile_t CLOISTER_TILE = {
  .name = "cloister",
  .traversal_cost =
     ONE_STEP_STASH,
  .opening_cost = {
    ONE_STEP_STASH,
    ONE_STEP_STASH,
    ONE_STEP_STASH,
    ONE_STEP_STASH
  },
  .weight = 2
};
```

FIGURE 2 – Exemple d'implémentation d'un cloister

2.3 Représentation du plateau de jeu

Le plateau de jeu est représenté par une matrice de pointeurs vers des structures `tile_t`. Chaque case du plateau fait référence donc à une tuile existante, sans en contenir une copie. Ce choix présente plusieurs avantages :

- il évite la duplication de structures potentiellement volumineuses
- il réduit la consommation mémoire
- il simplifie les opérations de lecture et de modification des tuiles
- il permet de comparer des tuiles via leurs adresses

On verra ceci en détail dans la section suivante.

2.4 Opérations sur les tuiles

Les différentes tuiles du jeu sont regroupées dans un tableau de pointeurs *p_tiles* vers les implémentations statiques. Cette organisation facilite l'implémentation des fonctions.

Les structures étant implémentées de manière statique et manipulées uniquement à travers des pointeurs, elles ne sont pas accessibles en dehors `tile.c`. Les interactions avec les tuiles (lecture des coûts, vérification des ouvertures, génération, etc.) sont donc réalisées à l'aide de fonctions dédiées, définies dans l'interface du module.

Nous présentons les opérations faites sur les tuiles prenant pour entrée un pointeur pointant sur une tuile statique de `tile.c`.

- `tile_name` : retourne une chaîne de caractères correspondant au nom de la tuile pointée par le pointeur passé en paramètre.

— `tile_traversal_cost` et `tile_opening_cost` : retournent la poche correspondant respectivement au coût de traversée ou au coût d'ouverture d'une tuile selon une direction donnée.
— `tile_generate_empty`, `tile_generate_start` et `tile_generate_exit` : retournent des pointeurs vers la tuile vide, la tuile de départ et la tuile de sortie, utilisés pour construire le manoir.
— `tile_generate_by_name` : retourne un pointeur vers la tuile dont le nom correspond à la chaîne de caractères passée en paramètre, en parcourant le tableau `p_tiles`.
— `tile_is_empty` : vérifie si le pointeur fourni désigne la tuile vide.
— `tile_can_open` : vérifie si une tuile peut être ouverte dans une direction donnée, en testant si le coût d'ouverture associé possède un nombre de *steps* strictement inférieur à `UINT_MAX`.

La fonction plus complexe est **tile_draft** :

Pour une direction en paramètre on retourne au maximum NUM_TILES_PER_TURN pointeur de tuile, avec un buffer, choisis de manière aléatoire pouvant être ouvert dans la direction en paramètre.

Avant de faire ça on appel au début du jeu une fonction `init_tile` qui initialise la graine aléatoire et initialise une matrice statique *p_tiles_direction*. Cette matrice contient quatre tableaux indexés selon les directions cardinales, chaque tableau contient toutes les pointeur de tuiles pouvant être ouvert sa direction associée. `p_tiles_direction[NORD]` est le tableau de pointeur de toutes les tuiles qui sont ouvertes au nord.

On place la tuile *weight* fois dans son tableau.

Dans la fonction `tile_draft` on répète NUM_TILES_PER_TURN fois :

On choisit un pointeur de tuile aléatoirement dans le tableau *p_tiles_direction* correspondant à la direction passé en paramètre. Si elle a déjà été choisie, c'est-à-dire déjà présente dans le buffer, alors on ne la remet pas, sinon ou la place dans le buffer.

En terme de complexité :

En temps, on vérifie à chaque fois si la tuile n'a pas été répétée, ce qui engendre une complexité de

$$\Theta(1 + 2 + ... + N) = \Theta(\frac{N(N-1)}{2}) = \Theta(N^2) \quad (1)$$

Avec N=NUM_TILES_PER_TURN. Le processus pour vérifier les doublons est une boucle dans une boucle.

On améliore la complexité en ne choisissant pas une tuile aléatoirement entre toutes les tuiles jusqu'à qu'elle s'ouvre dans la bonne direction mais si parmi *p_tiles_direction* associée. Néanmoins cela engendre la complexité spatiale liée aux quatres tableaux.

On a pas besoin de beaucoup s'en inquiéter ici car NUM_TILES_PER_TURN=3 et on a 7 types de tuiles ce qui ne sont des chiffres relativement petits et donc une complexité manageable. Mais on conclut que la complexité de ce module est liée au nombres de tuiles implémentées

On remarque que la quantité de tuiles choisie par la fonction est variable, pour cela on retourne le nombre de tuiles choisies.

3 Implémentation du jeu

À présent on possède le code nécessaire pour implémenter le jeu.

3.1 Manoir et joueur

On définit une structure qui permet de mémoriser l'état du jeu en tout moment.

```
struct game_state_t {
  struct stash_t steps;
  struct tile_t* board[BOARD_WIDTH][BOARD_HEIGHT];
  struct location_t current_location;
};
```

Code 2 – Structure du jeu

On garde dans cette structure les ressources du joueur dans la poche `steps` et sa position dans le manoir `current_location`.

Le manoir lui est un rectangle de taille (`BOARD_WIDTH, BOARD_HEIGHT`) composé d'adresses de tuiles comme abordé dans la section précédente. On stocke le type de tuile de chaque position dans la matrice `board`.

Avant de commencer la boucle de jeu, nous créerons une instance de cette structure appelée `game_state`. Elle permettra de définir l'état de notre jeu et sera le seul appel de cette structure.

3.2 Boucle du jeu

Après avoir défini l'état de notre jeu `game_state` on peut passer à la boucle de jeu, c'est-à-dire le déroulement de la partie.

```
init_tiles(t);
init_tiles_bis(t);
init_state(s, &game_state);
init_tui(is_graphic);

while (current_turn <= n && !is_player_won && game_state.steps.r[
   STEP] > 0)
{
    printf("\e[1mTurn number %d out of %d\e[0m\n", current_turn,
       n);
    int num_possible_locations =
       list_empty_locations_with_finite_opening_cost(
       possible_locations, &game_state);
    if (current_turn == 1 || (num_possible_locations &&
       decide_to_place_tile(is_iterative)))
    {
        printf("Place your tile\n");
        struct location_t selected_location = select_location(
           is_interactive, possible_locations,
           num_possible_locations);
        int num_available_tiles = generate_some_tiles(
           selected_location, available_tiles, &game_state);
        struct tile_t* selected_tile = select_tile(is_interactive
           , available_tiles, num_available_tiles);
        place_tile(selected_location, selected_tile, &game_state)
           ;
        tui_place_tile(is_graphic, selected_location,
           selected_tile);
    }
    else
    {
        printf("Move the player\n");
        int num_possible_locations = list_open_location(
           possible_locations, &game_state);
        struct location_t selected_location = select_location(
           is_interactive, possible_locations,
           num_possible_locations);
        tui_move_player(is_graphic, selected_location, &
           game_state);
        move_player(selected_location, &game_state);
    }
    current_turn++;
    tui_print(is_graphic);
    is_player_won = exit_reached(&game_state);
}
```


Code 3 – Implémentation de la boucle du jeu

Analysons l'implémentation du code 3

Avant de rentrer dans la boucle on initialise tous nos modules. En particulier on initialise `game_state`, c'est-à-dire : on positionne le joueur en $(0, 0)$; on pose toutes les tuiles du manoir comme vides sauf les tuiles en $(0, 0)$ qui est la tuile de départ et (`BOARD_WIDTH-1, BOARD_HEIGHT-1`) qui est la tuile d'arrivé ; on définit les ressources initiales du joueur de $4 * \frac{BOARD_SIZE}{3}$ steps.

Dans la boucle, à chaque itération, on vérifie que le nombre maximal de tours n'a pas été dépassé, que le joueur n'a pas encore gagné la partie et qu'il dispose de ressources suffisantes pour continuer à jouer ($step > 0$). Sinon la partie est terminée, le joueur à gagné s'il a atteint la tuile d'arrivée sinon perdu.

Pour chaque tour, `list_empty_locations_with_finite_opening_cost` cherche et stocke, dans un buffer, les localisations vides ouvrables depuis la position du joueur (adjacente et ouverte depuis la tuile du joueur). Si il en trouve au moins une, on donne au joueur la possibilité de placer une tuile dans une de ces positions ou bouger, sinon on l'oblige à bouger. Le joueur est obligé de placer une tuile le premier tour.

Si le joueur place

1. Le joueur choisit une des tuiles vides disponibles
2. `tile_draft` génère des tuiles compatibles à être placé à la place de la tuile vide dans `generate_some_tiles`
3. Le joueur choisit une de ces tuile
4. On remplace la tuile vide par la tuile choisie dans `game_state`. Le joueur paie les coûts associés.

Dans le jeu, on manipule des positions *location* alors que les fonctions du module *tile* travaillent avec des directions. Pour faire le lien, nous avons créé la fonction `location_to_dir`. Cette fonction prend la position actuelle du joueur et une position adjacente et retourne la direction de la dernière en comparant la position donnée avec les positions situées au nord, au sud, à l'est et à l'ouest du joueur à l'aide d'une structure $switch - case$.

Si le joueur bouge

1. On liste toutes les listes dont le joueur peut bouger, c'est-à-dire la tile du joueur et la tuile listée sont ouvertes une vers l'autre et adjacentes
2. Le joueur sélectionne une de ces tuiles
3. On bouge le joueur, update `game_state` et faisons payer le joueur les coûts associées.

Chaque "choix" peut se faire de manière aléatoire ou le joueur (derrière l'écran) peut faire les choix avec le mode itératif (expliqué plus bas).

On remarque que de nombreuses fonctions de la boucle principale nécessitent l'accès à l'état courant du jeu pour fonctionner correctement. Pour cette raison, l'adresse de la structure `game_state` est passée en paramètre à ces fonctions, ce qui leur permet d'accéder directement à l'ensemble des informations nécessaires.

Cette approche améliore la lisibilité et la maintenabilité du code : au lieu de transmettre séparément le manoir, la position du joueur ou ses ressources, l'intégralité de l'état du jeu est centralisée dans une seule structure.

De plus, le passage par adresse permet aux fonctions de modifier directement l'état du jeu et d'accéder aux données sans surcoût inutile. En particulier, la matrice board n'est pas recopiée à chaque appel de fonction ; elle est simplement lue ou modifiée via `game_state`, ce qui évite des opérations coûteuses et des complexités supplémentaires.

3.3 Exécution

3.3.1 Le Makefile

Un *Makefile* est un fichier utilisé par la commande *make* pour automatiser la compilation du projet selon des règles.

Notre Makefile a pour règles :

— `%.o: %.c`, règle générique qui compile chaque fichier source en fichier objet.
— `tester:` *fichier objet*, crée un exécutable capable d'exécuter les tests du projet
— `project:` *fichier objet*, crée le fichier de jeu exécutable à partir des fichier sources compilés en objet grâce à la première règle
— `test:` , exécute l'exécutable test *tester*
— `compile: project tester`, crée l'exécutable du jeu et des tests
— `all:` , règle compile

3.3.2 Paramètres d'exécution

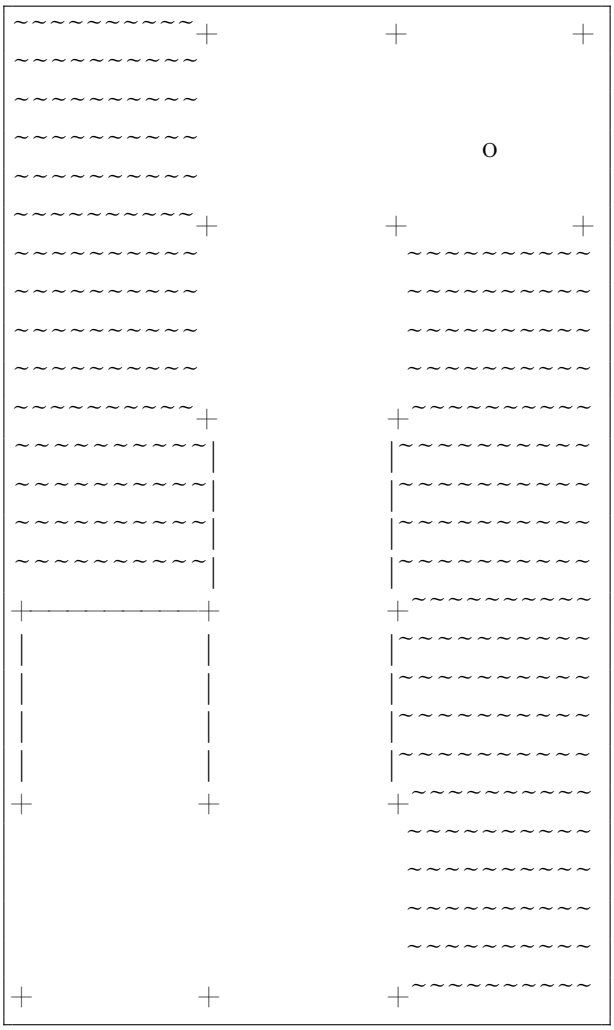
L'analyse de la ligne de commande pendant l'exécution se fait au moyen de la bibliothèque *getopt*, cela permet pour un même exécutable de changer les paramètres essentiels suivants :

— rendre le jeu interactif afin de pouvoir joueur : **-i**.
— graine aléatoire pour prendre les décision à la place du joueur (mode non interactif) : **-s**.
— graine aléatoire `tile_draft` : **-t**.
— fixer le nombre maximum de tours de jeu possible : **-n**.
— graine aléatoire pour les équipements : **-q**

Une autre fonctionnalité à été mise en place pour une meilleur expérience de jeu :

— visualisation graphique tui du manoir : **-g**

3.4 Visualisation graphique : TUI



Code 4 – TUI : visualisation du manoir

Une des fonctionnalités intéressantes à ajouter au projet était la TUI (Text User Interface), cet outil permet la visualisation du jeu directement dans le terminal. Le manoir est affiché sous forme d'une grille textuelle constitué des cases qui sont représentées sous la forme d'un tableau de caractères, l'implémentation consiste à initialiser le manoir rempli par les cases vides et de placer chaque tuile dans sa case tout en respectant ses directions d'ouverture avec une mise à jour au fur et à mesure de l'avancement du jeu. La TUI offre également une représentation de la position du joueur sur le plateau, mise à jour à chaque déplacement.

Cette interface ne gère pas la logique du jeu, mais fournit une vue cohérente de son

état courant, facilitant ainsi l'observation du déroulement de la partie et la validation du comportement du jeu lors des tests d'exécution. La figure ci-dessous illustre un exemple de la visualisation du manoir.

4 Limites et améliorations

Nous avons à présent un jeu fonctionnnel mais basique qui peut être ammélioré.

Dans la fonction `tile_draft`, nous avons souligné l'importance d'optimiser le nombre de tuiles différentes générées en terme de compléxité du jeu. Cependant, nous remaquons beaucoup d'implémentations redondantes de tuiles. On implémente 4 types terraces differentes et 2 couloires pour prendre en compte les différentes orientations possibles.

Également en terme de jouabiliter notre jeu peut être ammélioré.

Nous présenterons ici les modifications données à la version de base pour répondre aux limites actuelles du jeu. Nous finirons avec une nouvelle boucle de jeu et architecture.

4.1 Achievement 1 : Rotation des tuiles

Dans la version de base, l'implémentation des tuiles impliquait beaucoup de redondance et complexité.

Nous proposons une nouvelle approche associant à chaque type de tuile une rotation. Pour cela, nous définissons la structure `tile_bis_t`, contenant un pointeur vers une tuile et sa rotation, représentée par un type énuméré.

Lors de l'utilisation de ces tuiles, direction et rotation sont prises en compte simultanément grâce à `ROT_TO_DIR(d,r)`, qui convertit le couple (direction, rotation) en direction effective via la formule `(d + r) % MAX_DIRS`.

Cette structure est exploitée sur les nouvelles fonctions implémentées dans un nouveau fichier `tile_bis.c` pour des questions d'extensibilité.

`tile_opening_cost_bis` et `tile_can_open_bis` qui sont des adaptations des fonctions *tile* sont la structure `tile_bis_t` en tenant compte de la rotation.

`tile_draft_bis` retourne des *tile_bis* selon la direction et les ressources du joueur, en combinant tuiles déjà compatibles et tuiles aléatoires éventuellement pivotées pour satisfaire l'ouverture.

Finalement on introduit la fonction `random_tile_rotation`, qui permet au joueur de faire une rotation aléatoire compatible de la tuile choisie.

Dans l'implémentation du jeu on définit à présent le manoir comme une matrice de *tile_bis* qui utilisent uniquement les variantes de *tile_bis* des fonctions modifiées. De nos tuiles implémentées cette méthode permet de passer de 10 tuiles à 6 tuiles implémentées statiquement.

4.2 Achievement 2 : Tuiles objectives

Nous introduisons le concept de *target_tiles*(tuile objective). À présent pour rentrer dans la tuile d'arrivé et gagner la partie, on a besoin des ressources NUM_RESOURCES-2, NUM_RESOURCES-1 et NUM_RESOURCES ; pour NUM_RESOURCES=4 cela correspond a 1 or, 1 gem et 1 clé. Ces ressources sont réparties dans le manoir sur les *target_tiles*.

Chaque *target_tile* est associée à une unique "target ressource". Ainsi, pour gagner, il faut passer par tous les *target_tiles*.

4.2.1 Changement en game_state

Pour garder en mémoire les *target_tiles*, on ajoute dans `game_state` un tableau de localisation représentant les *target_tiles* : `target_locations`. Chaque indice du tableau est associée une *target_ressource* : `target_locations[0]` est la position où est situé l'or, `target_locations[1]` pour la gem et `target_locations[2]` pour la clé.

On ajoute également `is_target_locations_visited` tableau de booléens de qui indique, pour chaque indice, si son *target_tile* associée à été visité ou pas.

4.2.2 Modification de la boucle pour les `target_tiles`

Un fichier `target.c` a été ajouté pour gérer les `target_tiles`.

Dans la boucle de jeu, à chaque tour, les positions des `target_tiles` sont triées par proximité au joueur à l'aide d'un tri à bulles sur leurs distances de Manhattan à travers `sort_target_locations`. Les emplacements déjà visités sont placés à la fin, et seules les positions non visitées sont affichées.

De plus, lorsqu'un joueur se déplace sur un `target_tile`, les ressources correspondantes lui sont attribuées grâce à `target_location_reached` qui vérifie si c'est le cas puis met à jour ses ressources.

4.3 Ajouter une intelligence au joueur

Un problème du mode non-itératif est que le joueur effectue des actions totalement aléatoires, conduisant souvent à des parties perdues.

La solution est de créer un chemin qui approche le joueur de son objectif et le suivre.

Nous définissons `target_location_near` comme la position du `target_tile` non visité le plus proche, ou la tuile de sortie si toutes ont été visitées. Un chemin vers cette position est alors calculé à l'aide de `path_target_finder`.

4.3.1 path_target_finder

La fonction `path_target_finder` prend en paramètres la localisation objective, la position actuelle du joueur contenue dans `game_state`, ainsi qu'un buffer `path` qui est

un tableau de localisations destiné à stocker le chemin calculé.

L'objectif de cette fonction est d'atteindre la localisation cible ou, à défaut, une tuile vide la plus proche possible de celle-ci, en empruntant un chemin de longueur minimale. Cette approche permet d'approcher progressivement le joueur de son objectif en minimisant le nombre de tuiles à poser. Pour cela, la fonction s'appuie sur une méthode récursive, `path_target_finder_rec`, afin d'explorer les chemins possibles.

Avant de lancer la recherche récursive, les structures suivantes sont initialisées :

— `dist`, représentant la distance de Manhattan entre la dernière tuile du chemin courant (souvent une tuile vide) et la tuile objectif ;

— `path_len`, correspondant à la longueur du meilleur chemin trouvé ;

— `is_visited_position`, une matrice de la taille du manoir indiquant, pour chaque position, si elle a déjà été visitée afin d'éviter les cycles.

Ces variables sont ensuite passées par adresse à la fonction récursive et jouent le rôle de variables globales partagées entre tous les appels récursifs.

4.3.2 Fonction récursive de recherche de chemin

La fonction `path_target_finder_rec` réalise une exploration en profondeur du plateau à partir de la position courante. À chaque appel, la position est ajoutée au chemin en cours et marquée comme visitée.

Cas de bas : Si la position courante correspond à la localisation objectif, le chemin courant est enregistré comme solution optimale. Si la longueur du chemin dépasse la taille maximale du plateau, l'exploration est interrompue.

Recursion : Pour chaque direction possible, la fonction vérifie que la position suivante est valide, non visitée et accessible selon les règles d'ouverture des tuiles. Deux cas sont alors distingués :

— si la tuile suivante est vide, le chemin est évalué à l'aide de la distance de Manhattan à l'objectif et peut améliorer la meilleure solution courante

— si la tuile suivante est ouverte dans la direction opposée, la fonction s'appelle récursivement afin de poursuivre l'exploration

À la fin de l'exploration d'une position, celle-ci est dé-marquée, ce qui permet son utilisation dans d'autres chemins possibles.

4.3.3 Complexité

La fonction `path_target_finder` repose sur une exploration récursive en profondeur du plateau de jeu. Dans le pire des cas, chaque position du manoir est visitée au plus une fois grâce à la matrice `is_visited_position`, ce qui donne une complexité temporelle en O(BOARD_SIZE).

Cependant, en pratique, l'algorithme peut explorer plusieurs chemins partiels avant de converger vers une solution, ce qui rapproche son comportement d'un parcours exhaustif des chemins possibles. Le coût est donc exponentiel mais acceptable avec les contraintes du manoir.

La complexité spatiale est dominée par la matrice `is_visited_position` donc O(BOARD_SIZE).

4.3.4 Accommodation des changements dans la boucle de jeu

Après le processus, nous avons un chemin `path`. Il amène vers la localisation de la tuile objective ou une tuile vide qui approche le joueur de l'objectif.

Dans la boucle de jeu en crée le chemin puis déplaçons le joueur au long du chemin jusqu'à atteindre `target_location_near` ou une tuile vide.

Si on a atteint `target_location_near` on recommence le processus

Sinon, on est devant une tuile vide. On place une tuile et recommençons le processus.

La limitation de cette fonction est que nous n'avons pas de contrôle sur le type de tuile placée. Il est possible que le joueur s'enferme en plaçant des terraces par exemple. S' il n'existe pas de chemins vers un objectif alors on fini le jeu et perdons.

La boucle est affichée dans les annexes.

4.4 Achievement 3 : Équipements fonctionnnels

Cet achievement permet d'implémenter des équipements afin de permettre au joueur de modifier l'état du jeu, de façon à ce qu'il ait la possibilité de gagner différentes ressources selon l'équipement qu'il possède.

Les équipements fonctionnels sont définit par le type énuméré *enum*, il en existe quatre :

- la pelle : le joueur gagne, avec une probabilité 1 / BOARD_SIZE , une ressource aléatoire
- le porte-monnaie : à chaque fois que le joueur gagne une ressource GOLD, il gagne un GOLD supplémentaire
- les chaussures de course : le joueur a une chance sur trois de se rembourser un STEP pour chaque déplacement réalisé
- les chaupelles de monnaie : le joueur gagne un STEP à déplacement, ouvrement d'une tuile ou gain de ressources

Cet achievement repose sur l'implémentation d'une fonction *can_gain_equipement* avec laquelle le joueur a une chance sur BOARD_SIZE pour gagner un équipement aléatoire à chaque ouverture d'une tuile, ainsi que les fonctions *on_move*, *on_tile_open* et *on_resource_gain* qui permettent d'appliquer les équipements que le joueur possède. Puis insérer chacune de ses fonctions dans la boucle du jeu aux instants dédiés. une fonction qui permet l'initialisation la graine du générateur aléatoire et une fonction de

conversion d'un équipement en une chaîne de caractères pour informer le joueur s'il a gagné un équipement.

Ceci demande alors d'ajouter un tableau `has_equipement[MAX_EQUIPEMENTS]` dans `game_state` rempli des 1 et 0, qui indique si le joueur possède l'équipement associé à l'indice du tableau grâce au type énuméré des équipements.

4.5 Nouvelle boucle de jeu & architecture

Après les modifications, nous avons des nouveaux modules avec des nouvelles fonctions et donc nouvelle architecture et boucle.

Dans la nouvelle architecture, nous remarquons que *game* n'est plus un dépendant direct de la boucle et qu'il y a des modules intermédiaires pour inclure les achievements. Et *tile* n'est plus un dépendant directe de *game* mais passe par l'intermédiaire *tile_bis* pour ajouter les rotations.

On remarque que la boucle à été complètement restructurée pour afin de pouvoir suivre le chemin de `path_find`. Nous avons ajouté également les fonctions associés aux modules.

Nous avons quand même garder l'ancienne boucle avec l'inclusion des achivements, elle est accessible avec le parametre `-b`.

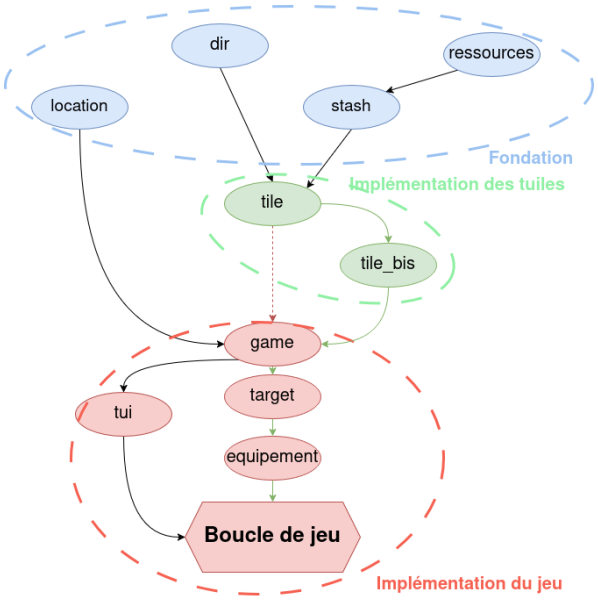


FIGURE 3 – Architecture après modifications

Remarques et conclusion

Remarques

Cette section présente les principales difficultés rencontrées au cours du projet, ainsi que les solutions mises en place pour y répondre.

Une première difficulté rencontrée lors du projet concernait la gestion de la rotation des tuiles dans l'achievement. Dans un premier temps, les rotations étaient définies séparément dans un fichier dédié, avec des fonctions permettant de consulter et de modifier la rotation d'une tuile.
Cette approche compliquait toutefois la logique du jeu, car elle nécessitait de maintenir deux tableaux distincts : un pour les tuiles et un autre pour leurs rotations. Ces tableaux devaient ensuite être initialisés et synchronisés dans le code principal, ce qui rendait la boucle de jeu plus lourde et plus difficile à maintenir.
Pour simplifier cette gestion, une seconde solution a été mise en place et qui consiste à définir une nouvelle structure de tuile regroupant à la fois la tuile et sa rotation. Cette solution permet de manipuler une tuile comme une seule entité, ce qui facilite son utilisation dans la boucle de jeu et rend le code plus clair.

C'était donc la raison pour laquelle on a un ordre d'achievements un peu bizarre et donc le bon ordre des achievements est :

1. Achiev0
2. Achiev1.2
3. Achiev1.6
4. Achiev2
5. Achiev3

Concernant le second achievement la fonction qui était un peu difficile à implémenter est la fonction récursive cherchant le chemin vers la tuile-objective, l'objectif principale était d'éviter une complexité trop élevée. Après réflexion, cette partie s'est révélée moins difficile que prévu, la solution étant surtout intuitive et naturelle à mettre en place.

Le troisième achievement était simple pour l'implémenter puis inclure les fonctions dédiés dans la boucle du jeu, sauf qu'on avait un problème concernant les ressources mais après utilisation de la fonction `stash_to_string` pour visualiser la progression des ressources on a conclu que la faute était tout simplement à cause d'une boucle.

Conclusion

Ce projet a permit de mettre en pratique les notions de programmation en langage C vues au cours du semestre, à travers la réalisation d'un jeu complet et fonctionnel. La conception modulaire du code a facilité la compréhension du projet et son évolution progressive.

Les différentes difficultés rencontrées ont été l'occasion de réfléchir à des choix d'implémentation plus adaptés et d'améliorer la structure du programme. Au final, le jeu obtenu respecte les objectifs du projet et constitue une base claire pour d'éventuelles améliorations.

Annexes

Nouvelle boucle du manoir

```
struct location_t path_to_target[BOARD_SIZE];
int is_game_over = 0;

while(!is_game_over)
{
  struct location_t target_location_near = (num_target_locations > 0)
    ? target_locations_sorted[0] : location(BOARD_WIDTH-1, BOARD_HEIGHT-1);
  unsigned int path_to_target_len = path_target_finder(target_location_near
&game_state, path_to_target);
  if (path_to_target_len >= BOARD_SIZE) { //no path existing
    printf("You have blocked yourself, you cannot reach your next objective\n);
    target_location_near = game_state.current_location;
    is_game_over = 1;
  }

  unsigned int i = 1;
  while(i < path_to_target_len && !tile_is_empty(location_to_tile(path_to_target[i],
&game_state).tile) && !is_game_over) {
    struct stash_t initial_resources = game_state.steps;
    printf("\e[1mTurn number %d out of %d\e[0m\n", current_turn, n);
    print_target_locations_sorted(target_locations_sorted, num_target_locations);
    print_path_target(path_to_target, path_to_target_len, target_location_near);
    struct location_t selected_location = path_to_target[i];
    printf(">> Move the player to the location (%d,%d)\n", selected_location.x,
  selected_location.y);

    tui_move_player(is_graphic, selected_location, &game_state);
    move_player(selected_location, &game_state);
    target_location_reached(&game_state);
    struct stash_t final_resources = game_state.steps;
    struct stash_t gained_resources = stash_sub(&final_resources, &initial_resources);
    on_move(&game_state);
```

```
    on_resource_gain(&game_state, gained_resources);
    i++;

    current_turn++;
    tui_print(is_graphic);
    is_player_won = exit_reached(&game_state);
    printf("\n");

    if (current_turn > n || is_player_won || game_state.steps.r[STEP] <= 0)
is_game_over = 1;
    }
  if (i < path_to_target_len && !is_game_over) {
    struct stash_t initial_resources2 = game_state.steps;
    printf("\e[1mTurn number %d out of %d\e[0m\n", current_turn, n);
    print_target_locations_sorted(target_locations_sorted, num_target_locations);
    print_path_target(path_to_target, path_to_target_len, target_location_near);
    struct location_t selected_location = path_to_target[path_to_target_len-1];
    printf(">> Place tile in the location (%d,%d)\n", selected_location.x,
  selected_location.y);

    enum dir_t d = location_to_dir(selected_location, &game_state);
    int num_available_tiles = tile_draft_bis(dir_opposite(d), game_state.steps,
  available_tiles);
    struct tile_bis_t selected_tile = select_tile(0, available_tiles,
  num_available_tiles);
    random_tile_rotation(0, selected_tile, d, &num_rotations);
    place_tile(selected_location, d, selected_tile, &game_state);
    can_gain_equipement(&game_state);
    struct stash_t final_resources = game_state.steps;
    struct stash_t gained_resources = stash_sub(&final_resources, &initial_resources2)
    on_tile_open(&game_state);
    on_resource_gain(&game_state, gained_resources);
    tui_place_tile(is_graphic, selected_location, selected_tile);

    current_turn++;
    tui_print(is_graphic);
    is_player_won = exit_reached(&game_state);
    printf("\n");
  }
  if (current_turn > n || is_player_won || game_state.steps.r[STEP] <= 0)
is_game_over = 1;
```

```
  num_target_locations = sort_target_locations(target_locations_sorted, &game_state);
}

}
```